

Autonomous Feature Pipeline Setup Guide

n8n + Claude Desktop + CloudCLI + Railway + GitHub + Telegram

Based on the tutorial: *I Built an Autonomous Feature Pipeline on n8n — GitHub Issues In, Shipped Code Out*

CONTENTS

1. Overview
2. Prerequisites & Accounts
3. CloudCLI Setup
4. Railway Setup
5. GitHub: Token & Label
6. Telegram Bot Setup
7. n8n Setup & MCP Connection
8. Claude Desktop Setup
9. Building the Workflow with Claude
10. Wiring Credentials in n8n
11. Activate & Test
12. The New Developer Loop

1. Overview

This guide walks you through the complete environment setup required to run an autonomous coding pipeline. Once active, the pipeline works as follows:

1. An n8n scheduler fires at 2 am every night (or on any interval you choose).
2. n8n pulls all open GitHub issues in your repository that carry the `ai-ready` label.
3. Each issue is dispatched one at a time to a CloudCLI coding agent session (Claude Opus 4.8). The agent reads your codebase, implements the change, and commits it.
4. n8n opens a draft pull request per issue, links back to the original issue, and posts the PR link and the CloudCLI session URL as a comment on the issue.
5. A Telegram message summarises the run: how many PRs were opened, any failures, and a link to the repo.

6. You wake up, review the diffs and session logs, and merge what looks correct. Railway detects the merge and auto-deploys.

The same Claude Opus 4.8 model that implements the code also built the n8n workflow itself — from a plain-English spec — using n8n's MCP tools.

2. Prerequisites & Accounts

You will need accounts and credentials from five services before building the workflow. Gather these first.

Service	Minimum Plan	What you need from it
Anthropic / Claude Desktop	Claude Pro	Desktop app + Opus 4.8 access
n8n	Free trial or self-hosted	Instance URL + MCP access token
CloudCLI	Hobby (from €7/mo) or free trial	API key + environment ID
Railway	Starter (free) or Pro	GitHub-linked project + autodeploy on
GitHub	Free	Personal access token + <code>ai-ready</code> label
Telegram	Free	Bot token + your chat ID

Note: CloudCLI provides the cloud environment — it does not include an AI subscription. You bring your own Anthropic API key or Claude Pro subscription; CloudCLI stores it encrypted inside the container.¹

3. CloudCLI Setup

CloudCLI is a cloud development environment purpose-built for AI coding agents. It runs Claude Code (and other agents) in a persistent container that stays alive even when your laptop is closed. n8n calls it via a REST API to kick off a coding session against your repo.¹

3.1 Create Your Account

- 1 Go to **cloudcli.ai** and click **Get Started Free** (no credit card required for the trial).¹
- 2 Sign up with your email or GitHub account.
- 3 Choose a plan. The **Hobby plan** (€7/month) gives you 1 active environment with 2 CPU cores and 4 GB RAM — sufficient for this pipeline. The **Growth plan** (€20/month) allows up to 5 active environments if you want parallel agent runs.¹

3.2 Create a Cloud Environment

- 1 From your CloudCLI dashboard, click **New Environment**.
- 2 Give it a descriptive name (e.g., `my-app-agent`).
- 3 Select the base image. The default Node/Python image is suitable for most web projects.
- 4 Click **Create**. The environment will provision in under a minute.
- 5 Note the **Environment ID** — you will paste this into the n8n workflow credential later.

Tip: CloudCLI environments are agent-agnostic. Claude Code, Cursor CLI, Codex, and Gemini CLI all come pre-installed — you choose which agent a session uses at call time via the API.¹

3.3 Connect Your GitHub Repository

- 1 Inside your new environment, open **Settings > GitHub Integration**.
- 2 Authorise CloudCLI to access your GitHub account.

- 3 Select the repository the coding agent will work against. CloudCLI clones it into the container so the agent has the full codebase available on every session start.

3.4 Add Your Anthropic API Key

- 1 In the environment settings, find **API Keys**.
- 2 Add your Anthropic API key. It is stored encrypted and never shared outside your container.¹
- 3 If you are on Claude Pro (subscription), you can alternatively authenticate via the Claude Code login flow inside the container terminal.

3.5 Get Your CloudCLI API Key

- 1 Open your CloudCLI **Account Settings** (top-right avatar menu).
- 2 Go to **Developer / API**.
- 3 Click **Generate API Key**. Copy it immediately and store it in a password manager — it will not be shown again in full.
- 4 The full API reference is available at **developer.cloudcli.ai**.²

Important: You will need two values for the n8n workflow: your **CloudCLI API Key** and your **Environment ID**. Keep both handy.

3.6 What the n8n Community Node Does

Instead of wiring up a generic HTTP Request node, this pipeline uses the official CloudCLI verified community node for n8n ([@cloudcli-ai/n8n-nodes-cloud-cli](#)). The node handles authentication, request formatting, and response parsing for you.⁷

The node exposes two resource types:

- **Environment** — Create, Delete, Get, Get Many, Start, Stop a cloud environment.
- **Agent** — Execute: run Claude Code, Cursor CLI, or Codex on a running environment and receive the session URL and output log in return.

The **Agent — Execute** operation is the one the pipeline uses. You supply the environment (selected from a dropdown populated by your API key), compose the prompt from the

GitHub issue, and the node returns the session URL that gets posted back to the issue as a comment. Installation is covered in Section 7.3.

4. Railway Setup

Railway is the deployment platform that picks up merged pull requests and redeploys your application automatically. Once linked to your GitHub repository, no further configuration is needed per-deploy — pushing to your main branch (or merging a PR into it) triggers a build and deploy.³

4.1 Create a Railway Account

- 1 Go to **railway.com** and sign up. Linking your GitHub account is the fastest option.
- 2 Railway offers a free Starter plan. The Pro plan is required for private Docker image deployments; most web app projects run fine on Starter.³

4.2 Create a New Project from GitHub

- 1 From the Railway dashboard, click **New Project**.³
- 2 Select **Deploy from GitHub repo**. Railway requires a linked GitHub account with contributor access to the target repository.⁴
- 3 Search for your repository and click it.
- 4 Choose **Deploy Now** to start building and deploying immediately, or **Add variables** if you need to set environment variables before the first deploy.³

4.3 Verify Auto-Deploy is Enabled

By default, Railway deploys on every push to the connected branch. This is the behaviour the pipeline relies on: when you merge a PR, Railway detects the new commit and starts a deploy automatically.⁴

- 1 Open your service on the Railway project canvas.
- 2 Go to **Service Settings > Source**.
- 3 Confirm the connected branch is your main branch (e.g., `main` or `master`).
- 4 Confirm **Auto-Deploy** is toggled on. If it was disabled, click **Enable**.⁴

Note: Autodeploy requires at least one project member to have a connected GitHub account with contributor access to the repository. If autodeploy cannot be re-enabled, verify the Railway GitHub App has access to the repo in your GitHub installation settings.⁴

4.4 Set Environment Variables

- 1 On the project canvas, click your service, then open the **Variables** tab.
- 2 Add any environment variables your app requires (database URLs, API keys, etc.).
- 3 Railway injects these at build and runtime automatically — no `.env` file committed to the repo is needed.

4.5 Generate a Public Domain (Optional)

- 1 After the first successful deploy, click your service on the canvas.
- 2 Open **Settings > Networking** and click **Generate Domain**.
- 3 Railway assigns a `.up.railway.app` subdomain immediately. You can attach a custom domain later.³

4.6 Install the Railway CLI (Optional)

The Railway CLI lets you deploy directly from a local terminal and is useful for debugging or one-off deploys outside the GitHub flow.⁵

Install

```
# macOS / Linux (via Homebrew)
brew install railway

# macOS / Linux (via shell script)
curl -fsSL https://railway.com/install.sh | sh

# Windows (via npm)
npm i -g @railway/cli
```

Authenticate

```
railway login
```

This opens a browser window to authorise the CLI with your Railway account.

Link to a project

```
cd your-project-directory  
railway link
```

Select your project and environment from the interactive prompt.

Deploy from the CLI

```
railway up
```

The CLI scans, compresses, and uploads your project files for deployment. Use `railway open` to jump to the project canvas in your browser.³

Tip: For this pipeline you do not need the CLI day-to-day — Railway picks up GitHub merges automatically. Install it only if you want a manual fallback or to test deploys locally.

5. GitHub: Personal Access Token & Label

5.1 Create a Personal Access Token

- 1 Go to **GitHub > Settings > Developer settings > Personal access tokens > Tokens (classic)**.
- 2 Click **Generate new token (classic)**.
- 3 Give it a descriptive name, e.g., `n8n-pipeline-token`.
- 4 Set an expiry (90 days or no expiry — your choice).
- 5 Select the following scopes:
 - `repo` — full control of private repositories (read issues, open PRs, post comments)
 - `workflow` — only needed if you want n8n to trigger GitHub Actions
- 6 Click **Generate token**. Copy and save it — GitHub shows it only once.

5.2 Create the ai-ready Label

- 1 Open your repository on GitHub.
- 2 Go to **Issues > Labels**.
- 3 Click **New label**.
- 4 Name it exactly `ai-ready` (the n8n workflow filters on this exact string).
- 5 Choose a colour and click **Create label**.

The discipline behind the label: Only label an issue `ai-ready` when the scope is clear, the expected behaviour is defined, and the context is sufficient for the agent to act without asking questions. Writing a good `ai-ready` issue is effectively writing a mini product requirements document.

6. Telegram Bot Setup

n8n sends an end-of-run summary to a Telegram bot you own. You need two values: the bot token and your personal chat ID.

6.1 Create a Telegram Bot

- 1 Open Telegram and search for **@BotFather**.
- 2 Send the command `/newbot`.
- 3 Follow the prompts: choose a display name, then a username (must end in `bot`, e.g., `my_pipeline_bot`).
- 4 BotFather replies with your **bot token** in the format `123456789:ABCDefgh...`. Copy and save it.

6.2 Get Your Chat ID

- 1 Send any message to your new bot (this starts the chat and makes your chat ID discoverable).
- 2 Open this URL in a browser, replacing `YOUR_BOT_TOKEN` :
`https://api.telegram.org/botYOUR_BOT_TOKEN/getUpdates`
- 3 In the JSON response, find the `"chat"` object and note the `"id"` value — this is your chat ID (a number, possibly negative for group chats).

7. n8n Setup & MCP Connection

7.1 Get an n8n Instance

You can use n8n in two ways for this pipeline:

- **Cloud (free trial):** Sign up at **n8n.io**. The free trial includes MCP server access. Your instance URL will be something like `https://yourname.app.n8n.cloud`.
- **Self-hosted:** Deploy n8n on Railway, a VPS, or Docker. The MCP server feature works on both cloud and self-hosted instances.

7.2 Enable the Instance-Level MCP Server

- 1 In your n8n instance, open **Settings** (left sidebar).
- 2 Scroll to **Instance-level MCP**.
- 3 Toggle on **Enable MCP access**. You must have owner or admin rights to see this option.
- 4 Click **Connection details**.
- 5 Copy the **MCP server URL**.
- 6 Click the **Access Token** tab and copy the token immediately. n8n shows the full token only once — if you close this panel and return, the copy button will be disabled.

Critical: Copy the access token before closing the Connection details panel. If you miss it, you will need to regenerate a new token.

7.3 Install the CloudCLI Community Node

The CloudCLI node is a verified community node — it is available directly inside n8n without needing npm or a terminal on n8n Cloud instances.⁷

On n8n Cloud

- 1 In your n8n instance, go to **Settings > Community Nodes**.

- 2 Click **Install**.
- 3 In the search field, type `cloudcli` — the verified package `@cloudcli-ai/n8n-nodes-cloud-cli` will appear in the results.⁷
- 4 Click **Install** next to the package. Agree to the community node usage terms if prompted.
- 5 The node is available immediately — no restart required.

On self-hosted n8n

If your n8n instance does not have GUI community node installation enabled, install the package manually from the command line inside your n8n environment:

```
npm install @cloudcli-ai/n8n-nodes-cloud-cli
```

Then restart n8n. The CloudCLI node will appear in the node panel under **Community Nodes** after the restart.

Tip: Once installed, you can find the node by typing "CloudCLI" in the n8n node search panel. It appears with an **Environment** and an **Agent** resource — use **Agent > Execute** for this pipeline.

Add the CloudCLI Credential

- 1 Click any CloudCLI node on the canvas (or add one temporarily from the node panel).
- 2 In the **Credential** field, click **Create new credential**.
- 3 Paste your **CloudCLI API Key** (generated in Section 3.5).
- 4 Click **Save**. n8n stores it encrypted and it will be available to all CloudCLI nodes in your instance.

8. Claude Desktop Setup

8.1 Download and Install

- 1 Go to **claude.ai/download** and download Claude Desktop for macOS or Windows.
- 2 Install and sign in with your Anthropic account.
- 3 You need a **Claude Pro** subscription to access Opus 4.8 from the model picker.

8.2 Select Claude Opus 4.8

- 1 Open a new chat in Claude Desktop.
- 2 Click the model name at the top of the chat window to open the model picker.
- 3 Select **Claude Opus 4.8**.

8.3 Connect Claude Desktop to n8n via MCP

This is a two-step process: no config file editing, no terminal commands, no restart required.

- 1 In Claude Desktop, open **Settings > Connectors** (or **Integrations** depending on your version).
- 2 Click **Add custom connector**.
- 3 Paste the **MCP server URL** you copied from n8n.
- 4 When prompted for authentication, paste the **access token**.
- 5 Click **Save** or **Connect**.

You will see the n8n connector appear as active. A tools icon will appear in the Claude Desktop chat input bar, confirming that Claude can now call n8n's MCP tools directly.

9. Building the Workflow with Claude

9.1 Generate the Build Spec First

Before asking Claude to build the workflow, ask it to write a PRD (product requirements document) from a plain English description. Building from a spec means Claude implements something that already exists on paper, rather than inferring architecture from scratch.

In Claude Desktop (with Opus 4.8 selected), type:

```
I want to build an n8n workflow that runs every night at 2am, finds all open GitHub issues in my repo labeled "ai-ready", sends each one to CloudCLI to implement, opens a draft PR per issue linking back to the original issue, comments on the issue with the PR link and the CloudCLI session URL, and sends me a Telegram summary when it's done. Write a PRD for this workflow that I can use as a build spec.
```

Claude will produce a node-by-node breakdown including scheduling, batching, error handling, and credential requirements. Review it before the next step.

9.2 Trigger the Build

Once you are happy with the spec, send:

```
Using the n8n MCP tools available to you, build this workflow exactly as specified: [paste the PRD here]
```

Claude will begin making tool calls in sequence:

1. **get_sdk_reference** — reads the n8n Workflow SDK before writing a single line of code.
2. **search_nodes** queries — confirms the exact node types exist (GitHub, Schedule Trigger, SplitInBatches, CloudCLI, Telegram) and pulls their correct parameter definitions. Because you installed the CloudCLI community node in Section 7.3, it will appear in the node inventory and Claude can reference it by name.
3. Writes TypeScript against the n8n Workflow SDK — each node declared with the correct schema, connections explicit. This is n8n's current workflow creation path; type errors are caught at compilation, not on the canvas.

4. **validate_workflow** — validates before creating anything. If there is a field-level error, Opus 4.8 reads the exact failure message, adjusts the TypeScript, and re-validates rather than retrying blindly.
5. **create_workflow_from_code** — creates the workflow on your n8n instance, returning a workflow ID and a direct URL to the canvas.

Switch to your n8n tab and refresh. You will see the eight-node workflow fully connected on the canvas.

9.3 What Claude Builds

Node	Purpose
Schedule Trigger	Cron fires at 2 am nightly (configurable)
GitHub — List Issues	Pulls all open issues with the <code>ai-ready</code> label
SplitInBatches	Processes one issue at a time (batch size 1) to avoid API rate limits and conflicting agent sessions
Set	Builds the prompt for CloudCLI: issue title, body, and instructions to implement the minimum change following existing code conventions
CloudCLI — Agent Execute	Runs Claude Code on your selected environment using the composed prompt. Returns the session URL and full output log. Uses the <code>@cloudcli-ai/n8n-nodes-cloud-cli</code> community node — no raw HTTP wiring required. ⁷
GitHub — Create PR	Opens a draft PR with <code>closes #issue-number</code> in the body so GitHub links them automatically
GitHub — Comment	Posts the PR link and CloudCLI session URL on the original issue
Telegram	Sends end-of-run summary: PRs opened, failures, repo link

10. Wiring Credentials in n8n

Claude leaves placeholder credential fields in the workflow. Open the n8n canvas and fill each one in. The CloudCLI API key is stored as a dedicated node credential (set up in Section 7.3) — the environment is selected from a dropdown inside the node itself, not entered as a separate credential field.

Node	Credential / Setting	Where to find it
GitHub nodes	GitHub Token	Generated in Section 5.1
CloudCLI — Agent Execute	CloudCLI API Key (node credential)	Created in Section 7.3 — select it from the credential dropdown
CloudCLI — Agent Execute	Environment (in-node setting)	Select your environment from the dropdown — populated automatically from your API key
Telegram	Bot Token	Provided by @BotFather (Section 6.1)
Telegram	Chat ID	Retrieved via the getUpdates API call (Section 6.2)

To add credentials in n8n: click the relevant node on the canvas, click the credential field, choose **Create new credential**, and paste the value. n8n stores credentials encrypted — they are never embedded in the workflow JSON.

Estimated time: With all credentials gathered in advance, filling these fields takes approximately 15 minutes from workflow import to a fully active pipeline.

11. Activate & Test

11.1 Test the Workflow Manually

- 1 In n8n, open the workflow canvas.
- 2 Create a test GitHub issue in your repo and add the `ai-ready` label.
- 3 Click **Test workflow** (the play button) to trigger a manual run without waiting for the scheduler.
- 4 Watch the nodes execute left to right. Green checkmarks indicate success; red indicates an error — click the node to read the error detail.
- 5 Check GitHub: a draft PR should appear, and the original issue should have a comment with the PR link and CloudCLI session URL.
- 6 Check Telegram: you should receive a summary message.

11.2 Activate for Scheduled Runs

- 1 Once the test run is clean, click the **Activate** toggle in the top-right of the n8n canvas.
- 2 The workflow is now live. The scheduler will fire at 2 am on the schedule you set.

11.3 Review and Merge

- 1 When you wake up, check Telegram for the run summary.
- 2 Open each draft PR in GitHub. Read the diff — the code changes are fully reviewable.
- 3 Click the CloudCLI session URL posted in the issue comment to open the live session log and audit every file the agent touched and every decision it made.
- 4 If the implementation is correct, click **Merge**.
- 5 Railway detects the new commit on your main branch and starts a deploy automatically. The feature is live.

You are not approving a black box. The session log shows the agent's complete reasoning trace: every file read, every change made, every decision taken. Merge only what you would merge from a human PR.

12. The New Developer Loop

Running this pipeline consistently introduces a discipline shift that improves all your development work, not just the automated portion.

Specify. Build. Verify.

Writing the issue *is* the job. A good `ai-ready` issue has clear scope, defined expected behaviour, and enough context for the agent to act without asking questions. This is effectively a mini product requirements document. The agent's output quality is a direct reflection of the input quality.

You become the gatekeeper

You are no longer a scheduler or executor. Hook your issue intake to Slack or email — new requirements arrive, become draft issues automatically, and await your review. You tighten the scope, confirm the acceptance criteria, and apply the `ai-ready` label when the issue is truly ready. That is the whole interaction.

The scheduler can run more frequently

The 2 am schedule is a starting point. Change the cron to run every hour. Requirements arrive during the day, get clarified, get labeled, get built — all before end of day. Adjust the Schedule Trigger node in n8n to whatever cadence suits your workflow.

Writing better issues makes everything better

The side effect of this discipline is that even the issues you plan to implement yourself become clearer and better-scoped. The quality bar for `ai-ready` issues raises the quality bar for all issues. That is the compounding return on the initial setup investment.

REFERENCES

1. CloudCLI — Cloud Dev Environments for AI Coding Agents. <https://cloudcli.ai/>
2. CloudCLI API Documentation. <https://developer.cloudcli.ai/>
3. Railway Quick Start Tutorial. <https://docs.railway.com/quick-start>
4. Railway — Controlling GitHub Autodeploys. <https://docs.railway.com/deployments/github-autodeploys>
5. Railway — Deploying with the CLI. <https://docs.railway.com/cli/deploying>
6. n8n Workflow — Start an AI coding agent from Linear issues with CloudCLI. <https://n8n.io/workflows/14071-start-an-ai-coding-agent-from-linear-issues-with-cloudcli/>
7. @cloudcli-ai/n8n-nodes-cloud-cli — npm package (verified n8n community node). <https://www.npmjs.com/package/@cloudcli-ai/n8n-nodes-cloud-cli>